Two-Dimensional Graph Renderer Specification

Physical and schematic layouts, Matplotlib rendering, and diagnostic decluttering

mdstats

Contents

1 Purpose and status

This document is the normative specification for `mdstats.plotting.graph_2d` in `mdstats` 0.11.0. It owns two-dimensional layout, Matplotlib artist construction, backward-compatible renderer-local periodic controls, explicit G4 consumption, and the `GraphRenderResult` presentation record.

2 Motive

Two-dimensional figures remain the primary backend for static diagnostics, annotation, vector export, and publication. The renderer must produce clean, reproducible figures while making graph omissions, projection choices, and periodic display conventions explicit.

3 Normative ownership

This specification owns:

- `GraphLayoutOptions`;
- `Graph2DRenderOptions`;
- `GraphRenderResult`;
- `plot_decorated_graph_2d()`;
- physical and schematic 2-D layout algorithms;
- projected edge paths, ghost endpoint artists, axes, labels, and legends;
- renderer-level complexity enforcement and warnings.

It consumes `DecoratedGraphView`, selection contracts, and `GraphStyle`; it does not redefine them.

4 AI context summary

- Physical and schematic layouts are semantically distinct.
- `auto` uses physical coordinates when available and spring layout otherwise.
- PCA orientation must be deterministic.
- Periodic edge paths are display geometry, not graph identity.
- Local unwrapping is display-only and may leave residual winding on non-tree edges.
- Ghost endpoints are faded artists, not graph nodes.
- Parallel-edge overlap is rejected by default.
- Labels are off by default and complexity-accounted.
- Every render returns the actual projected geometry and selection metadata.

5 Layout options

5.1 `GraphLayoutOptions`

```
@dataclass(frozen=True, slots=True)
```

```
class GraphLayoutOptions:
    method: Literal[
        "auto",
        "physical",
        "spring",
        "circular",
        "shell",
    ] = "auto"

    projection: Literal[
        "xy", "xz", "yz", "pca"
    ] | NDArray[np.float64] = "pca"

    center_physical: bool = True
    seed: int = 0
    spring_iterations: int = 100
    spring_k: float | None = None
```

Selection rules:

- `auto` chooses `physical` when physical coordinates exist;
- otherwise `auto` chooses `spring`;
- `physical` requires `node_positions_3d`;
- schematic methods ignore physical coordinates for node placement;
- the selected method is recorded in render metadata.

A custom projection matrix must have shape `(2,3)`, finite entries, rank 2, and nonzero row norms.

5.2 Physical projections

Cartesian projections are

$$P_{xy} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad P_{xz} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix},$$

and similarly for P_{yz}.

5.3 PCA projection

For positions $\mathbf{r}_i$, define centered coordinates

$$X_i = \mathbf{r}_i - \bar{\mathbf{r}}.$$

The first two right-singular vectors of X define the projection plane. Component signs must be stabilized by making the largest-magnitude Cartesian component of each vector positive.

Near-degenerate singular values make the in-plane orientation physically arbitrary. The renderer should emit a warning when the two leading directions are numerically degenerate. The output remains deterministic under the documented tie-breaking rule.

5.4 Schematic layouts

Spring, circular, and shell layouts are schematic. They should use NetworkX when available. `seed` must control spring-layout determinism.

If NetworkX is unavailable, physical rendering remains available, while schematic layout requests raise an optional-dependency error.

6 Two-dimensional renderer options

6.1 `Graph2DRenderOptions`

```python
@dataclass(frozen=True, slots=True)
class Graph2DRenderOptions:
    figsize: tuple[float, float] | None = None
    dpi: int = 150
    show_axes: bool | None = None
    equal_aspect: bool = True
    margin_fraction: float = 0.06
    title: str | None = None
    tight_layout: bool = True
    periodic_edge_mode: Literal[
        "translated_segment",
        "canonical_quotient",
    ] = "translated_segment"
    periodic_node_mode: Literal[
        "canonical",
        "local_unwrapped",
    ] = "canonical"
    show_periodic_ghosts: bool = True
    allow_parallel_overlap: bool = False
```

Rules:

- positive `dpi` is required;
- figure dimensions must be positive;
- margins must be nonnegative;
- `show_axes=None` means axes are shown for physical layouts and hidden for schematic layouts;
- `translated_segment` draws a periodic edge to the translated target position;
- `canonical_quotient` connects canonical endpoint positions and marks the edge as periodic through style/metadata;
- the selected periodic edge and node modes must be recorded;
- `local_unwrapped` is meaningful only for physical layouts and is ignored with a warning for schematic layouts;
- `show_periodic_ghosts=True` draws faded endpoint replicas only for residual translated periodic edges;
- ghost endpoints are artists, not scientific nodes, and do not appear in `rendered_node_keys`;
- parallel edges require `allow_parallel_overlap=True` in the first renderer;
- self-loops are rejected in the first renderer.

`translated_segment` is physically correct for each residual periodic edge. The implemented renderer may place canonical nodes in a deterministic locally unwrapped image gauge and may draw faded target-image ghost endpoints. It does not yet create an expanded supercell or a public graph of display replicas; those remain deferred to G4.

7 `GraphRenderResult`

```python
@dataclass(slots=True)
class GraphRenderResult:
    figure: matplotlib.figure.Figure
    axes: matplotlib.axes.Axes

    rendered_node_keys: tuple[GraphKey, ...]
    rendered_edge_keys: tuple[GraphKey, ...]

    node_positions_2d: NDArray[np.float64]
    edge_paths_2d: tuple[NDArray[np.float64], ...]
```

```
    artist_groups: Mapping[
        str,
        tuple[matplotlib.artist.Artist, ...],
    ]

    layout_metadata: Mapping[str, Any]
    style_metadata: Mapping[str, Any]
    selection_metadata: Mapping[str, Any]
    periodic_metadata: Mapping[str, Any]
    complexity: GraphComplexityReport
    warnings: tuple[str, ...]
```

The result is intentionally mutable because Matplotlib figures and artists are mutable. The numeric arrays and metadata stored in it should nevertheless be copied and read-only where practical.

The result is not a scientific graph and must not be used for topology identity.

8 Public generic rendering function

```
def plot_decorated_graph_2d(
    view: DecoratedGraphView,
    *,
    layout: GraphLayoutOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    periodic: PeriodicDisplayOptions | None = None,
    options: Graph2DRenderOptions | None = None,
    axes: matplotlib.axes.Axes | None = None,
) -> GraphRenderResult:
    ...
```

`None` selects a documented default configuration. No mutable configuration object is used as a function default.

If `axes` is supplied, the renderer draws into that axes and ignores `figsize`. Otherwise it creates a new figure and axes.

The function does not save a file automatically. Users may call

```
result.figure.savefig("graph.svg")
```

or use a future convenience export helper. Matplotlib determines the output format from the extension.

9 Rendering pipeline

`plot_decorated_graph_2d()` must execute the following conceptual stages:

1. validate public inputs
2. resolve focus on the original graph
3. apply explicit node and edge filters
4. create private PreparedGraphView
5. estimate render complexity
6. enforce complexity policy
7. resolve whether the layout is physical or schematic
8. prepare physical periodic node images and residual edge shifts when requested
9. resolve physical or schematic node positions
10. construct edge paths and optional display-only periodic ghost endpoints
11. resolve node and edge styles
12. batch Matplotlib artists by compatible style

13. add selected labels and legend
14. set limits, aspect, axes, title, and background
15. return GraphRenderResult

Every stage must be deterministic for the same inputs and dependency versions.

10 Layout and edge-path algorithms

10.1 Physical node projection

Physical node positions are projected using the selected matrix P:

$$X^{2D} = X^{3D} P^{\mathsf{T}}.$$

When `center_physical=True`, the projected centroid is translated to the origin. No rotation beyond the selected projection and no scale normalization is applied.

10.2 Local periodic node placement

For `periodic_node_mode="local_unwrapped"`, the renderer assigns each node an integer display-image offset $\mathbf{q}_i$ using a deterministic spanning forest. For an oriented tree edge $i \to j$ with display shift $\mathbf{m}_{ij}$,

$$\mathbf{q}_j = \mathbf{q}_i + \mathbf{m}_{ij}.$$

The displayed node coordinate is

$$\tilde{\mathbf{r}}_i = \mathbf{r}_i + \mathbf{q}_i H.$$

Every edge then carries the residual shift

$$\widetilde{\mathbf{m}}_{ij} = \mathbf{m}_{ij} + \mathbf{q}_i - \mathbf{q}_j.$$

Tree edges have zero residual shift. Non-tree edges may retain nonzero residual shifts when they encode periodic winding. The renderer must preserve that winding; it must not force all edges into one Euclidean image.

When `show_periodic_ghosts=True`, each nonzero residual edge may draw a faded copy of its translated target endpoint. These ghost points are display-only artists. They are deduplicated by target node and residual shift, excluded from scientific node counts, and reported separately in `periodic_metadata`.

10.3 Physical periodic edge paths

For `translated_segment`, edge $e = (i, j, \mathbf{m})$ is represented by the two-point path

$$\left[P\mathbf{r}_i, P(\mathbf{r}_j + \mathbf{m}H)\right].$$

For `canonical_quotient`, it is represented by

$$\left[P\mathbf{r}_i, P\mathbf{r}_j\right],$$

and periodic status remains available for styling or labels.

10.4 Schematic edge paths

Schematic layouts connect the two assigned node coordinates. Periodic image shifts are not interpreted geometrically in schematic mode, but remain in metadata. `periodic_node_mode` is therefore not applicable to schematic layouts; a nondefault request is ignored with a warning and recorded in `periodic_metadata`.

10.5 Endpoint color transitions

For a straight edge from $\mathbf{x}_i$ to $\mathbf{x}_j$,

$$\mathbf{x}(t) = (1-t)\mathbf{x}_i + t\mathbf{x}_j.$$

For segmented gradients with endpoint colors $\mathbf{c}_i$ and $\mathbf{c}_j$,

$$\mathbf{c}(t) = (1-t)\mathbf{c}_i + t\mathbf{c}_j.$$

`midpoint_split` uses two line segments and is the default atomic-edge mode because it is visually clear and produces smaller vector files than a finely segmented gradient.

11 Matplotlib rendering strategy

The renderer should batch objects by resolved style rather than creating one artist per ordinary graph object.

Recommended artist types:

```
nodes                PathCollection from scatter
constant edges       LineCollection
segmented gradients  LineCollection over short segments
labels               Text or Annotation artists
highlight halos      separate PathCollection
```

Separate artist groups are permitted for different markers, line styles, transition statuses, or z-orders.

The renderer must preserve the mapping from artist groups back to scientific keys in `GraphRenderResult` metadata.

11.1 Node-display modes

The renderer consumes `GraphStyle.node_display_mode` after normal style-rule resolution.

- `markers`: render resolved `NodeStyle` objects normally.
- `dots`: preserve resolved face color and alpha, force a circular marker, use `GraphStyle.node_dot_size`, and remove the outline.
- `hidden`: create no node or periodic-ghost `PathCollection`; suppress node labels and node legend entries.

Hidden nodes remain in `rendered_node_keys`, `node_positions_2d`, edge endpoint indices, and selection metadata. Node-overlap warnings are suppressed in hidden mode because no node markers are visible. Edge geometry and endpoint-derived edge colors are unchanged.

12 Decluttering requirements

The first renderer must prioritize readable output rather than attempting to show all metadata simultaneously.

12.1 Default visual hierarchy

```
ordinary edges      low z-order and moderate transparency
highlight edges     greater width and opacity
nodes               above edges with contrasting outlines
selected nodes      larger marker or halo
labels              highest z-order
```

12.2 Labels

Labels are disabled by default. Publication and diagnostic users should label only selected nodes or small focused subgraphs.

12.3 Focus and filters

Graph-hop focus is the primary decluttering mechanism. Species, component, transition status, and explicit-key filters provide secondary control.

12.4 Axes and scale

Physical layouts should preserve equal scale by default. Schematic layouts should hide numerical axes by default.

12.5 Legends

`legend="auto"` should include only meaningful categorical mappings that are visible in the rendered graph, such as species or transition status. Duplicate legend entries must be removed. Hidden node mode must omit species-only node legend entries.

12.6 Projection overlap

A clean two-dimensional projection cannot always exist. Distinct nodes may overlap under any selected plane. The renderer should warn when projected node separation is small relative to the plot scale. It must not perturb physical positions silently.

Schematic layouts may be used when a topology-focused view is more important than physical geometry.

13 Performance expectations

Let N and M be selected node and edge counts.

Expected costs are:

Operation	Expected cost
Key validation	$O(N + M)$ average
Focus breadth-first search	$O(N + M)$
Attribute filtering	$O(N + M)$
Physical projection	$O(N + M)$
PCA projection	$O(N)$ for fixed dimension
Style resolution	$O(NR_N + MR_E)$ in the simple rule model
Matplotlib batching	$O(N + M + S)$

Here R_N and R_E are node and edge rule counts, and S is the number of gradient segments.

The implementation should group categorical rules where practical, but correctness and deterministic behavior take precedence over premature optimization.

14 Current periodic-display ownership

Generalized periodic materialization is owned by G4 in `periodic_graph.py`. The 2-D renderer accepts an explicit `periodic=` option, consumes the resulting `PeriodicGraphView`, and performs only projection and Matplotlib rendering.

The legacy 0.10.0 controls remain compatibility syntax:

- canonical wrapped node positions;

- deterministic local unwrapping over a spanning forest;
- translated periodic edge segments;
- canonical quotient segments;
- faded ghost endpoints for residual nonzero shifts.

The compatibility path reuses G4 deterministic image-assignment helpers. New canonical-cell ghost materialization, expanded-cell replication, and generalized source/display mappings must use `PeriodicDisplayOptions`.

15 Error and warning policy

Raise:

- `GraphLayoutError` for invalid layout or render options and failed projections;
- `GraphComplexityError` when an explicit complexity policy rejects the render;
- `GraphUnsupportedFeatureError` for unsupported self-loops or overlapping parallel edges under the selected policy;
- `GraphStyleError` when style resolution fails.

Warnings must be returned in `GraphRenderResult.warnings` and may also be emitted through Python warnings when appropriate. Relevant warnings include PCA degeneracy, projection overlap, ignored periodic options under schematic layouts, residual periodic winding after local unwrapping, and explicit `warn_and_render` complexity overflow.

16 Input constraints and edge cases

- Physical layout requires `node_positions_3d`.
- Translated periodic segments require a cell and a physical projection.
- Self-loops are not implemented in this renderer.
- Parallel endpoint pairs are rejected unless overlap is explicitly allowed.
- A dense three-dimensional graph can remain cluttered under any 2-D projection. Orthographic comparisons and focused subgraphs are preferred diagnostics.
- PCA axes can be unstable for nearly degenerate point clouds; the renderer must report this.
- Extremely small or collinear coordinate sets require finite fallback limits.

17 Testing requirements

Tests must cover:

- `xy`, `xz`, `yz`, and deterministic PCA projections;
- deterministic spring layout for a fixed seed;
- circular and shell layouts;
- automatic physical-versus-schematic selection;
- local unwrapping and residual winding metadata;
- translated and quotient periodic edge paths;
- ghost endpoints excluded from rendered scientific keys;
- node and edge artist counts;
- dot-mode marker sizes and colors;
- hidden-mode suppression of nodes, ghosts, node labels, and node legend entries;
- label and legend behavior;
- PNG, SVG, and PDF export;
- complexity rejection and explicit warn-and-render behavior;
- unsupported self-loop and parallel-edge handling;
- no mutation of the source view or style.

18 Public exports

`GraphLayoutOptions`
`Graph2DRenderOptions`
`GraphRenderResult`
`plot_decorated_graph_2d`

19 Future compatibility

The 3-D renderer consumes the same graph-view and style layers while retaining its own renderer options and result type. Matplotlib-specific fields must not be moved into the common graph view or style model.

20 G4 integration contract

Generalized periodic materialization is owned by `periodic_graph.py` and the Periodic Graph Display Specification. In `mdstats` 0.11.0, `plot_decorated_graph_2d()` accepts an explicit `periodic=` argument and renders the returned materialized display graph. The legacy local-unwrapping path preserves the 0.10.0 result contract but delegates deterministic image assignment to the G4 private helper rather than maintaining a separate BFS implementation. The existing fields

`periodic_edge_mode`
`periodic_node_mode`
`show_periodic_ghosts`

may remain as compatibility controls during migration, but their behavior must be translated to G4 option objects and documented as compatibility syntax. New expanded cell behavior must not be added directly to `graph_2d.py`.

The explicit `periodic: PeriodicDisplayOptions | None = None` argument is implemented. When it is supplied, nondefault legacy periodic controls conflict and raise `GraphLayoutError` rather than being resolved silently.

The 2-D renderer continues to own projection, Matplotlib artists, labels, legends, and static export. G4 owns display-node replication, local image assignment, winding ghosts, and source-to-display mappings.